

ReinforcedLN:
a Reinforcement Learning Approach for a Light Novel Recommender System

F415299
Institute of Digital Technologies

MSc - LLP131 Prototype

February, 2025

Contents

1 Introduction . . . 3
2 System Overview . . . 3
3 Data Handling and Preprocessing . . . 3
4 Reinforcement Learning Recommender . . . 4
4.1 Neural Network Structure . . . 4
4.2 Recommendation Agent . . . 4
5 User Interface and Feedback System . . . 5
5.1 Interactive Recommendation Process . . . 5
5.2 Visual Feedback Examples . . . 6
5.3 Testing and Feedback Trials . . . 7
6 System Integration . . . 8
6.1 Role of Large Language Models (LLMs) . . . 8
7 Evaluation and Visualisation . . . 8
7.1 Reward Progression . . . 8
7.2 Loss Reduction . . . 9
8 Conclusion . . . 9
References . . . 10

1. Introduction

This report presents a comprehensive overview of a prototype that employs a reinforcement learning (RL) approach combined with neural networks to deliver personalised light novel recommendations. The system, developed as a Minimum Viable Product (MVP), centres on an interactive Gradio user interface (UI) (Gradio-Team 2019) that accepts user-provided synopses and returns recommendations—including a title, synopsis, and cover image—while adapting its performance based on feedback. This prototype represents an individual contribution and focuses on a demonstrable aspect of the overall solution. The code for the prototype is also available to view on GitHub (Islam 2025).

2. System Overview

The prototype integrates multiple modules to form a complete pipeline:

- **Data Handling and Preprocessing**: Utilises *data_loader.py* to load a reduced dataset of light novels, merge genres and synopses into a unified text representation, and compute SBERT embeddings (Reimers and Gurevych 2019).
- **Reinforcement Learning Recommender**: Implements a neural network that scores candidate recommendations based on concatenated embeddings from both the user and the candidate light novels.
- **User Interface and Feedback System**: Uses Gradio to interact with the user, present recommendations, and capture feedback for real-time model training.
- **Evaluation and Visualisation**: Provides graphical representations of the reward progression and loss reduction during training, offering insights into the learning behaviour of the RL agent.

3. Data Handling and Preprocessing

data_loader.py is responsible for loading and preprocessing a CSV file that contains a curated set of light novels. It combines the 'Genres' and 'Synopsis' fields into a single text representation, ensuring that any missing data is managed appropriately. The module then uses the SBERT model ("all-MiniLM-L6-v2") to compute embeddings for each novel. These embeddings are crucial as they serve as the foundation for the recommendation process by providing a dense, semantic representation of each light novel.

4. Reinforcement Learning Recommender

rl_recommender.py is central to the system's ability to adapt based on user interactions. It consists of two primary parts:

4.1. Neural Network Structure

The RL recommender employs a neural network, RLRecommender, which scores candidate recommendations by evaluating the concatenated vector formed from a user embedding and a candidate's embedding. The architecture is shown in Fig.1

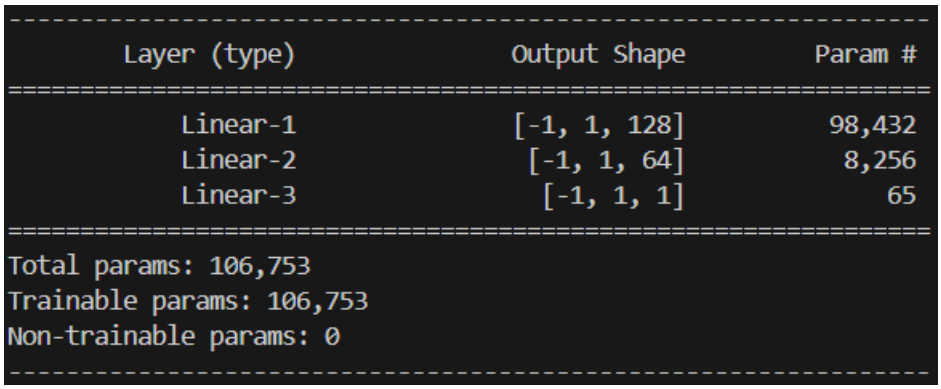


Fig 1: Neural Network Structure.

This structure consists of three fully connected layers. The first layer transforms the input (which is twice the embedding dimension) into a 128-dimensional feature space. The second layer further reduces this representation to 64 dimensions, and the final layer outputs a single scalar value that represents the recommendation score. The absence of non-trainable parameters indicates that every weight in the network is optimized during training, ensuring full adaptability based on user feedback.

4.2. Recommendation Agent

Building on the neural network, the *RecommendationAgent* applies an epsilon-greedy strategy for exploration (Wunder, Littman, and Babes 2010), ensuring that while the system exploits known high-scoring candidates, it occasionally selects random recommendations to discover potentially better options. Moreover, a diversity penalty is applied to recently recommended items to avoid

repetitive suggestions. The agent continuously updates its model through feedback provided by the user, adjusting the weights based on a simple reward scheme (+1 for positive and -1 for negative feedback).

5. User Interface and Feedback System

The user interface is built in *ui_feedback.py* using Gradio and is a critical element in showcasing the prototype's functionality:

5.1. Interactive Recommendation Process

Users are prompted to enter a synopsis, which is then embedded using SBERT. The RL agent processes this embedding against the dataset to generate a list of recommendations. The UI displays the details of the top recommendation, including:

- **Title and Synopsis**: Presented as formatted text.
- **Cover Image**: Fetched and displayed after processing the provided URL.

5.2. Visual Feedback Examples

To illustrate the effectiveness of the recommendations, two types of images are displayed:

- **Good Recommendation** (Fig.2): An image showcasing a light novel that received positive feedback, reflecting a good-quality match between the user input and the recommendation.

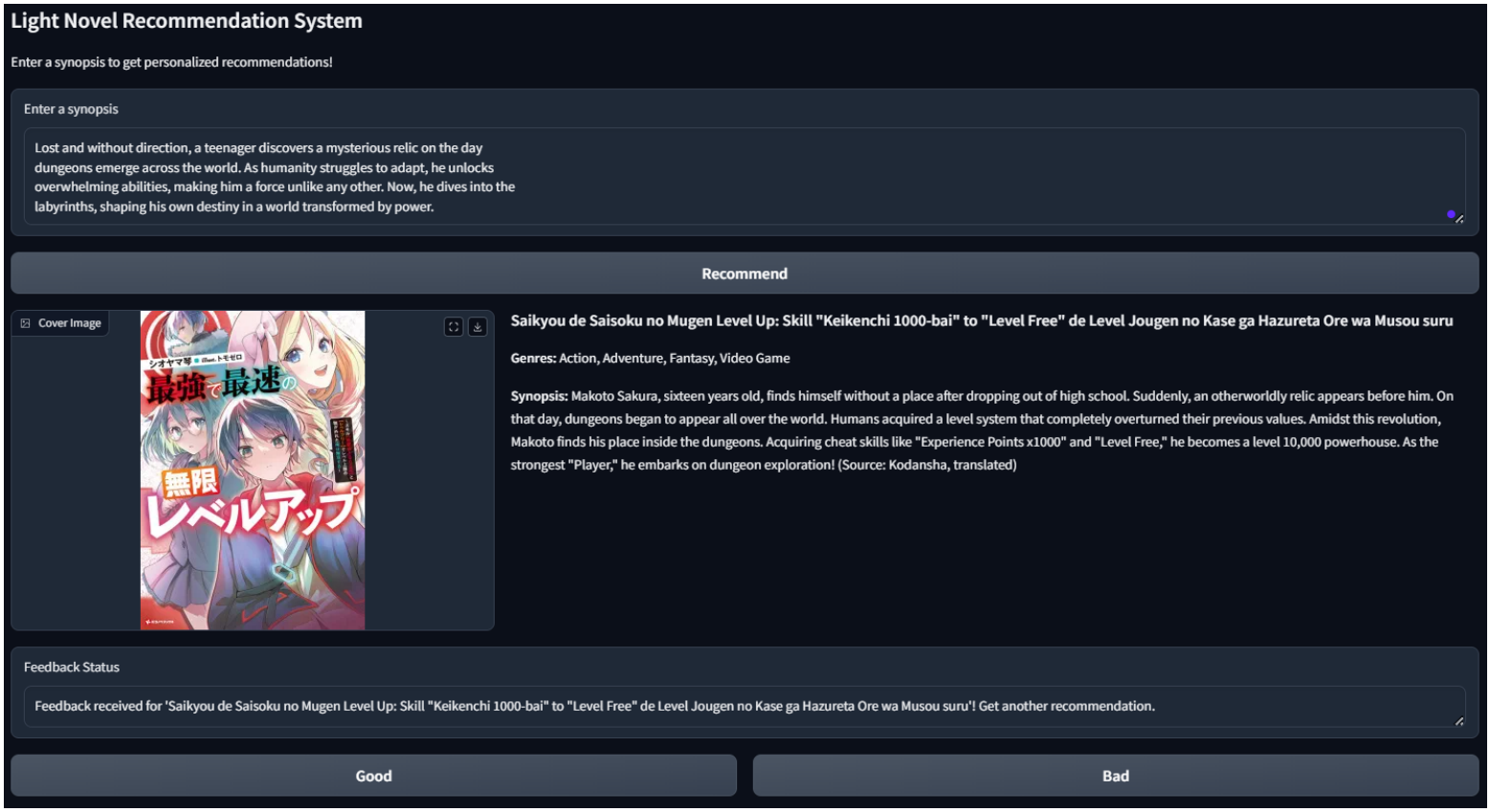


Fig 2: Good Recommendation.

- **Bad Recommendation** (Fig.3): An image corresponding to a recommendation that received negative feedback, triggering adjustments in the agent's scoring mechanism.

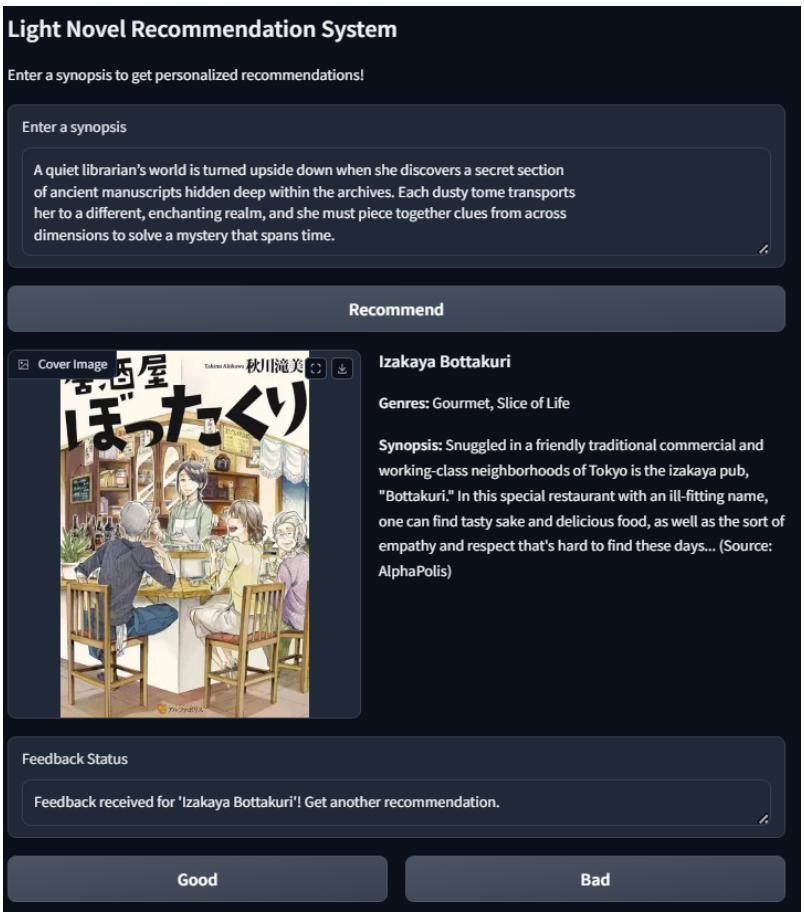


Fig 3: Bad Recommendation.

5.3. Testing and Feedback Trials

A series of tests were conducted using six different synopses of varying lengths to ensure robustness across a diverse range of inputs. For each synopsis, five recommendations were generated and corresponding feedback (good or bad) was collected, leading to 30 feedback trials. This systematic approach not only provides a broad evaluation of the system's performance but also supplies ample training data for the RL agent, fostering more accurate and personalized recommendations over time.

6. System Integration

The *main.py* module manages the flow of the system by loading and preprocessing the dataset using *data_loader.py*, initializing the RL-based recommender with *rl_recommender.py*, and launching the interactive Gradio UI from *ui_feedback.py*. This ensures seamless integration from data ingestion to recommendation and user feedback collection.

6.1. Role of Large Language Models (LLMs)

LLMs, specifically SBERT, are integral to the recommendation process. They generate embeddings for user-input synopses and the light-novel dataset, capturing semantic meaning beyond simple keyword matching. This allows the RL agent to compare user preferences with existing light novels effectively, leading to more contextually accurate recommendations.

7. Evaluation and Visualisation

By using *evaluation_visuals.py* Evaluation of the MVP focuses on two key metrics: reward progression and loss reduction over time.

7.1. Reward Progression

A graphical plot of the reward progression (Fig.4) illustrates how the system's performance evolves with user feedback. Rewards are recorded on a scale from -1 (indicating a poor recommendation) to 1 (indicating a good recommendation). The fluctuations in this graph reveal the dynamic nature of the RL agent as it balances exploration and exploitation. Over the 30 feedback trials, the rewards demonstrate an adaptive behaviour, with the agent gradually favouring recommendations that align more closely with user preferences.

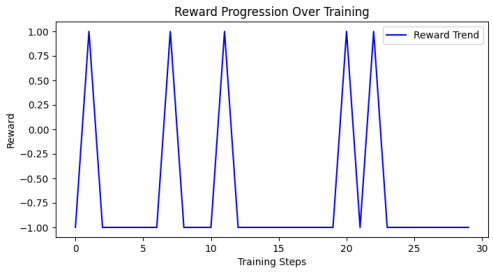


Fig 4: Rewards change over time. -1=Bad, 1=Good.

7.2. Loss Reduction

The loss reduction graph (Fig.5) provides insights into the training efficiency of the neural network. Starting from an initial loss value of approximately 0.95, the loss steadily decreases to around 0.43 after 30 feedback trials. Although there are occasional spikes in the loss—likely due to the inherent variability in user feedback—the overall downward trend confirms that the model is effectively learning and converging. These visualisations display the system's capability to refine its recommendations through continuous learning.



Fig 5: Loss reduction over time.

8. Conclusion

This prototype effectively demonstrates a reinforcement learning-based approach to personalised light novel recommendations. By combining advanced preprocessing, a robust RL agent, an interactive interface, and thorough evaluation, the system adapts to user feedback, validating the proposed solution. Structured testing with diverse synopses and visual feedback confirms its effectiveness, laying a strong foundation for future enhancements.

References

Gradio-Team (2019). *Gradio Documentation*. www.gradio.app. URL: https://www.gradio.app/docs.

Islam, M. T. (2025). *ReinforcedLN*. GitHub. URL: https://github.com/n30-Joker/ReinforcedLN (visited on 02/28/2025).

Reimers, N. and I. Gurevych (Aug. 2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. URL: https://arxiv.org/pdf/1908.10084.

Wunder, M., M. Littman, and M. Babes (2010). *Classes of Multiagent Q-learning Dynamics with epsilon-greedy Exploration*. URL: http://engr.case.edu/ray_soumya/mlrg/2010%20Classes%20of%20Multiagent%20Q-learning%20Dynamics%20with%20e-greedy%20Exploration.pdf.